

Manual Técnico: Contacto 3xtrat3rr3str3D

Fredy José Gabriel Herrera Funes

Este documento detalla la arquitectura, herramientas y el flujo interno del compilador Contacto 3xtrat3rr3str3D, diseñado para procesar tres lenguajes (Pig Latin, Y?, y Zetariano) mediante un entorno de desarrollo integrado (IDE) propio. El sistema realiza análisis léxico, sintáctico y semántico, culminando en la traducción del proyecto a código de tres direcciones en C (main.c).

1. Tecnologías Utilizadas

El compilador está construido sobre un stack moderno de Java, utilizando herramientas estándar para la generación de analizadores y la gestión del proyecto.

- **Java (OpenJDK):** Versión 21. Lenguaje base. Implementa características modernas como record y switch con patrones.
- **Apache Maven:** Versión 3.8+. Orquestación de construcción, gestión de dependencias y ejecución del proyecto.
- **ANTLR 4:** Versión 4.13.2. Generación de analizadores léxicos y sintácticos a partir de gramáticas .g4.
- **Java Swing:** JDK 21. Desarrollo de la interfaz gráfica de la IDE.
- **GCC:** Compatible C99. Compilación externa del archivo main.c generado.
- **draw.io:** Elaboración del diagrama de clases (docs/diagrama_clases.drawio).
- **Git / GitHub:** Control de versiones del código fuente.

Nota de Diseño: No se integran librerías externas para el coloreado de código. El resaltado de sintaxis en la IDE se ejecuta en tiempo real aprovechando los lexers autogenerados por ANTLR.

1.1 Configuración de Plugins (Maven)

- **maven-compiler-plugin (3.13.0):** Garantiza la compilación utilizando release 21.
- **antlr4-maven-plugin (4.13.2):** Procesa los archivos .g4 y genera los parsers con la opción visitor activada.
- **exec-maven-plugin (3.5.1):** Permite el lanzamiento directo de la IDE mediante mvn exec:java.

1.2 Instrucciones de Compilación y Ejecución

Para inicializar el entorno y la IDE desde la terminal:

mvn clean compile
mvn exec:java
Para compilar y ejecutar el código de tres direcciones (C3D) generado:

```
gcc main.c -o programa && ./programa
```

2. Distribución y Arquitectura del Proyecto

El código fuente está estructurado bajo el estándar de Maven, separando las gramáticas de ANTLR de la lógica abstracta en Java.

Descripción de Paquetes Core

- **compiler**: Núcleo de orquestación. Detecta el lenguaje por extensión, ejecuta el parser correspondiente, traduce los errores de ANTLR al español y carga la estructura del proyecto resolviendo las importaciones a partir del archivo .pig.
- **compiler.constructores**: Implementa los Visitors de ANTLR encargados de transformar el árbol de derivación nativo de ANTLR en un AST estandarizado.
- **compiler.ast**: Contiene las clases base (Nodo, Tipo, Programa). Define los contratos principales para semántica y traducción.
- **compiler.enviroment**: Maneja el estado de la memoria en tiempo de compilación mediante TablaSimbolos, ámbitos anidados y registros estructurales.
- **compiler.semantic**: Motor de reglas. Ejecuta las tres pasadas del análisis semántico y evalúa reglas de TablaCompatibilidad.
- **compiler.c3d**: Orquesta la traducción final (GeneradorC3D, TraductorC). Produce objetos Cuarteta e inyecta las rutinas de la clase Nativas.
- **gui.resaltado**: Motor de análisis léxico en tiempo real para la interfaz gráfica.

3. Flujo Interno del Compilador

El ciclo de vida de la compilación sigue un modelo lineal estricto, donde un error en cualquier fase detiene la propagación a la siguiente.

1. **Carga y Resolución**: Inicia con el archivo .pig principal. Resuelve recursivamente las instrucciones import basándose en rutas relativas.
2. **Análisis Léxico**: El lexer ignora comentarios. Caracteres no definidos disparan un error léxico.
3. **Análisis Sintáctico**: ANTLR construye el árbol de derivación aplicando estrategias de recuperación de errores.

4. **Construcción del AST:** Los Visitors consolidan los árboles derivativos en una jerarquía común.
5. **Análisis Semántico (3 Pasadas):**
 - **Pasada 1:** Registro de firmas globales.
 - **Pasada 2:** Validación estructural (atributos, firmas, sobrecargas).
 - **Pasada 3:** Verificación profunda de cuerpos (tipos, ámbitos).
6. **Generación C3D:** Si la tabla de errores está vacía, se inician los recorridos sobre el AST para emitir el código de bajo nivel.

4. Análisis Semántico y Sistema de Tipos

- **Tabla de Símbolos:** Estructurada jerárquicamente.
- **Jerarquía Numérica:** El sistema promueve tipos estáticamente en el orden carácter -> entero -> decimal.
- **Lectura y Conversión:** Las instrucciones de entrada detectan el tipo y convierten el texto.
- **Restricciones de Nomenclatura:** Está prohibido declarar una estructura y una clase con el mismo nombre.
- **Nulabilidad (Zetariano):** Exclusivo de Zetariano, el tipo String y las instancias de objetos pueden ser null. Los tipos primitivos no.

5. Generación de Código de Tres Direcciones (C3D)

La salida final se abstrae en el formato Cuarteta(operador, arg1, arg2, resultado). La clase TraductorC convierte este flujo en código C99 estandarizado.

- **Marcos de Función (Stack):** Se reservan posiciones para el valor de retorno, el contexto this, parámetros y variables locales.
- **Llamadas y Recursión:** El entorno guarda el estado de todos los temporales activos en el stack, simulando el cambio de ámbito.
- **Cadenas (Strings):** Almacenadas secuencialmente en el arreglo global heap.
- **Arreglos:** Proyectados aplanados en el heap. Se inserta una cabecera inicial con el tamaño estático.
- **Estructuras vs Objetos:** En Y? y Pig Latin, operan por copia de valor. En Zetariano, los objetos residen en el heap.
- **Control de Flujo:** Gestionado estrictamente por saltos absolutos y condicionales (if ... goto L; / goto L;).